SLIIT UNI
THE KNOWLEDGE UNIVERSITY

Faculty of Computing

Year 2 Semester 1 (2026)

IT2011 - Artificial Intelligence and Machine Learning

Lab Sheet 06

Unsupervised Learning

Learning Objectives

- Implement K-Means clustering to group data points based on similarity in feature space.
- Apply Hierarchical Clustering to identify data groupings and visualize the clustering process using a dendrogram.
- Evaluate clustering performance using the Elbow Method to determine the optimal number of clusters.
- Gain hands-on experience with Python libraries such as pandas, scikit-learn, matplotlib, and scipy for data analysis and visualization.

Introduction

Unsupervised learning is a branch of machine learning where models identify patterns or structures in data without labeled outcomes. This lab sheet focuses on two popular unsupervised learning techniques: K-Means Clustering and Hierarchical Clustering. You will use the Iris dataset to explore these methods, learning how to preprocess data, apply clustering algorithms, visualize results, and evaluate clustering quality.

The exercises emphasize practical implementation in Python, using libraries like pandas for data handling, scikit-learn for clustering, and matplotlib for visualization. By the end of this lab sheet, you will be able to perform clustering tasks and interpret their outcomes effectively.

Dataset

- **Source:** Iris dataset, classic dataset in machine learning, available through scikit-learn
- **Description:** 150 samples of iris flowers, each described by four features:
  - sepal length
  - sepal width
  - petal length
  - petal width (all measured in centimeters)

Task 1 – K-Means Clustering

1. Import the necessary Python libraries for K-Means clustering

Import libraries for data handling (pandas), clustering (scikit-learn's KMeans), visualization (matplotlib), and dataset loading (from sklearn.datasets import load iris).

2. Load a dataset using pandas and display its first five rows

Load the Iris dataset into a pandas DataFrame and preview the first few rows to understand the data structure.

3. Select the relevant features for clustering

Choose a subset of features (e.g., sepal length and sepal width) for clustering, as unsupervised methods work on feature spaces. This simplifies visualization to 2D.

```
features = df[['sepal length (cm)', 'sepal width (cm)']]
```

4. Visualize the dataset using a scatter plot

Create a scatter plot of the selected features to observe the data distribution and potential clusters visually before applying any algorithm.

```
plt.scatter(features['sepal length (cm)'], features['sepal width (cm)'])
plt.xlabel('Sepal Length (cm)')
plt.ylabel('Sepal Width (cm)')
plt.title('Dataset Scatter Plot')
plt.show()
```

5. Create and fit a K-Means clustering model with a chosen number of clusters

Initialize a KMeans model with an initial guess for the number of clusters (e.g., 3) and fit it to the selected features to group the data points.

```
kmeans = KMeans(n_clusters=3, random_state=42)
kmeans.fit(features)
```

6. Retrieve the cluster labels assigned to each data point

After fitting, extract the cluster assignments for each data point, which indicate which cluster each row belongs to.

```
labels = kmeans.labels_
print(labels)
```

7. Add these cluster labels as a new column to the dataset

Append the cluster labels to the original DataFrame as a new column for easy reference and further analysis.

```
df['cluster'] = labels
print(df.head())
```

8. Visualize the clusters and their centroids using a scatter plot

Plot the data points colored by their cluster labels and mark the centroids to show the clustering results visually.

```
plt.scatter(features['sepal length (cm)'], features['sepal width (cm)'], c=labels, cmap='viridis')
centroids = kmeans.cluster_centers_
plt.scatter(centroids[:, 0], centroids[:, 1], marker='X', s=200, c='red', label='Centroids')
plt.xlabel('Sepal Length (cm)')
plt.ylabel('Sepal Width (cm)')
plt.title('K-Means Clusters and Centroids')
plt.legend()
plt.show()
```

9. Calculate the Within-Cluster Sum of Squares (WCSS) for different numbers of clusters

Compute WCSS (inertia) for a range of cluster numbers (1 to 10) to measure how tightly data points are grouped within clusters.

```
wcss = []
for i in range(1, 11):
    kmeans = KMeans(n_clusters=i, random_state=42)
    kmeans.fit(features)
    wcss.append(kmeans.inertia_)
print(wcss)
```

10. Use the Elbow Method to determine the optimal number of clusters

Plot the WCSS values against the number of clusters to identify the ”elbow” point, where adding more clusters yields diminishing returns.

```
plt.plot(range(1, 11), wcss, marker='o')
plt.xlabel('Number of Clusters')
plt.ylabel('WCSS')
plt.title('Elbow Method for Optimal Clusters')
plt.show()
# Inspect the plot; the elbow is often at 3 for this dataset.
```

11. Re-train the K-Means model with the optimal number of clusters and plot the final clusters

Based on the elbow method, refit the model and visualize the final clusters with centroids.

```
optimal_k = 3  # Adjust based on elbow plot
kmeans_optimal = KMeans(n_clusters=optimal_k, random_state=42)
kmeans_optimal.fit(features)
labels_optimal = kmeans_optimal.labels_
df['optimal_cluster'] = labels_optimal
```

```
plt.scatter(features['sepal length (cm)'], features['sepal width (cm)'], c=labels_optimal, cmap='viridis')
centroids_optimal = kmeans_optimal.cluster_centers_
plt.scatter(centroids_optimal[:, 0], centroids_optimal[:, 1], marker='X', s=200, c='red', label='Centroids')
plt.xlabel('Sepal Length (cm)')
plt.ylabel('Sepal Width (cm)')
plt.title('Final K-Means Clusters')
plt.legend()
plt.show()
```

Task 2 – Hierarchical Clustering

12. Import the required Python libraries for hierarchical clustering and visualization

Import libraries for data handling (pandas), clustering (scikit-learn's KMeans), visualization (matplotlib), and dataset loading (from sklearn.datasets import load iris).

Import additional libraries for hierarchical clustering (AgglomerativeClustering and linkage from scikit-learn) and dendrogram visualization (scipy).

```
from sklearn.cluster import AgglomerativeClustering
from scipy.cluster.hierarchy import dendrogram, linkage
```

13. Load the dataset using pandas and display its first few rows

Load the Iris dataset as a DataFrame and preview it.

14. Select the relevant features for clustering

Choose a subset of features (e.g., sepal length and sepal width) for clustering.

15. Plot a dendrogram to visualize the hierarchical clustering process

Compute the linkage matrix using Ward's method and plot a dendrogram to show the hierarchical merging of clusters.

```
linked = linkage(features, method='ward')
plt.figure(figsize=(10, 7))
dendrogram(linked, orientation='top', distance_sort='descending', show_leaf_counts=True)
plt.title('Dendrogram for Hierarchical Clustering')
plt.show()
# Use the dendrogram to estimate the number of clusters
# (e.g., cut at a suitable height: suggesting 3 clusters ).
```

16. Perform Agglomerative Hierarchical Clustering with a chosen number of clusters

Initialize and fit an AgglomerativeClustering model with a chosen number of clusters (e.g., 3) using Ward linkage.

```
hc = AgglomerativeClustering(n_clusters=3, linkage='ward')
hc.fit(features)
```

17. Retrieve the cluster labels assigned to each data point

Extract the cluster assignments from the fitted model for each data point. (Similar to step 6 in Task 1)

18. Add the hierarchical cluster labels as a new column to the dataset

Append the hierarchical labels to the DataFrame for reference. (Similar to step 7 in Task 1)

19. Visualize the resulting clusters with different colors on a scatter plot

Plot the data points colored by hierarchical cluster labels to see the grouping.

```
plt.scatter(features['sepal length (cm)'], features['sepal width (cm)'], c=hc_labels, cmap='viridis')
plt.xlabel('Sepal Length (cm)')
plt.ylabel('Sepal Width (cm)')
plt.title('Hierarchical Clustering Results')
plt.show()
```

20. Compare the clusters obtained from hierarchical clustering with those from K-Means

Visualize both clustering results side-by-side to compare label assignments between K-Means and Hierarchical methods.

Analyze the plots on your own by comparing the cluster shapes, boundaries, and assignments. Consider similarities and differences in how the algorithms group the data points.

Draw your own conclusions about which method might be more suitable for this dataset and why, based on your observations (e.g., compactness of clusters, handling of outliers, or overall structure).